Viva Guide

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

Viva Guide

A companion to ARCHITECTURE.md: a 60-second summary, a screen tour, a suggested live-demo script, how the system meets the project requirements, and likely questions with honest answers.

> All screenshots here are generated from **sample data** (the API is stubbed), so every screen is shown fully populated.

---

1. The 60-second summary

Hostel maintenance at the University of Zambia used to be reported verbally or on paper: no tracking, no accountability, no data. This project replaces that with a Progressive Web App:

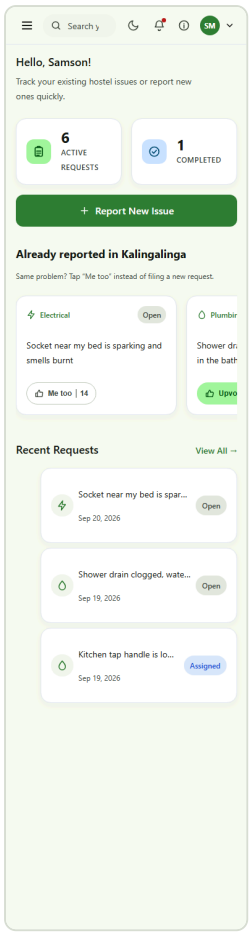
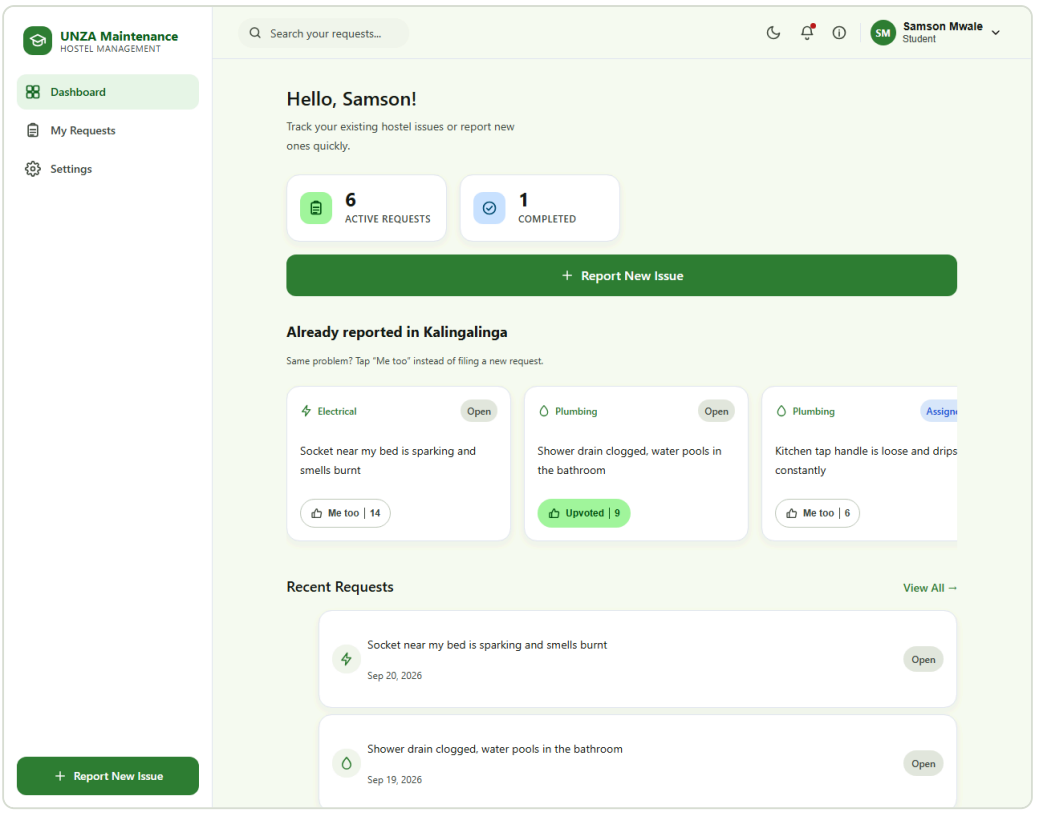
- **Students** report a fault from their phone (works offline), back existing reports instead of duplicating them, and watch the status update live.
- **Administrators** see everything, get overdue and workload warnings, assign the best-suited technician in one tap, and study analytics.
- **Technicians** get their tasks, update status, and can attach a photo and note of the finished work as proof.

Technically: a **React PWA → Express REST API (routes / services / repositories) → Supabase (PostgreSQL + Auth + Storage)**. The client never touches the database; every rule is enforced on the server.

---

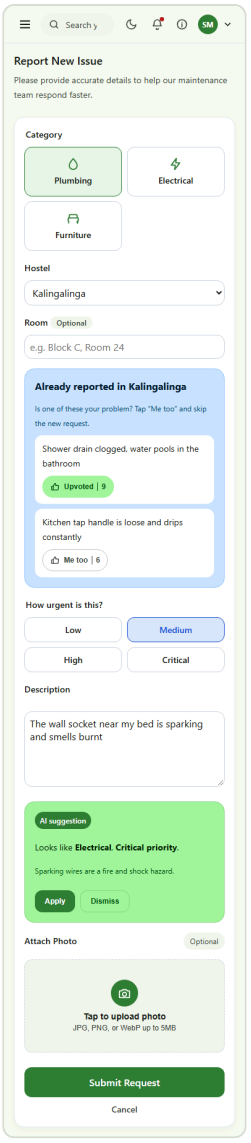
2. Screen tour

Student

Dashboard (phone)	Dashboard (desktop)
	

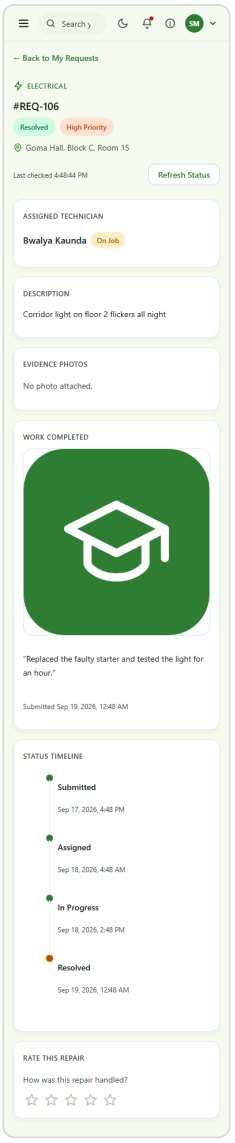
The **"Already reported in <hostel>"** strip lets students back an existing issue with *Me too* instead of filing a duplicate. The count tells administrators how many people a fault affects. Other students' names are never shown.

Reporting an issue



- **Duplicate warning** once category + hostel are chosen ("Already reported in…").
- **AI/rules suggestion** of category and priority while typing. Hazards such as sparking wires are *always* forced to critical by a safety rule, whatever the model says.
- Works **offline**: the request is queued on the device and submitted when connectivity returns.

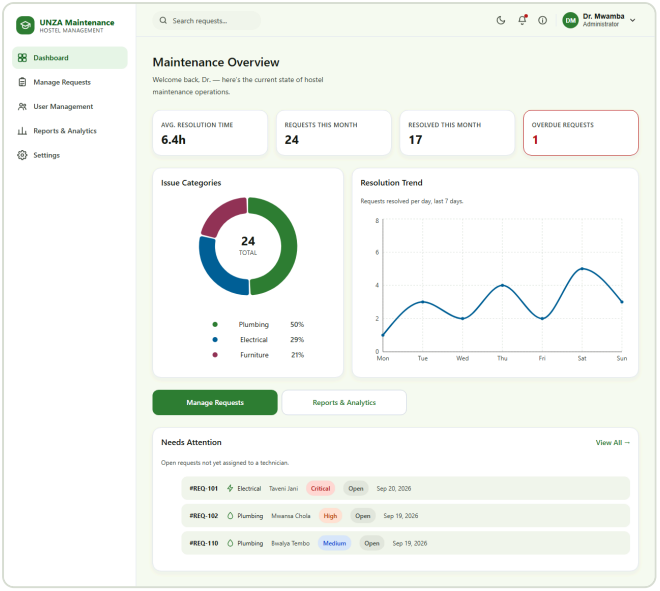
Request detail — "Work Completed"



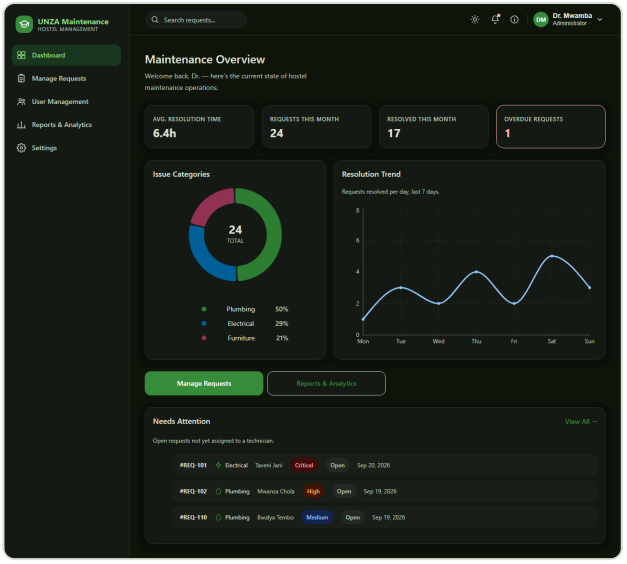
The student sees the technician's photo and note, the status timeline, and can rate the repair.

Administrator

Dashboard

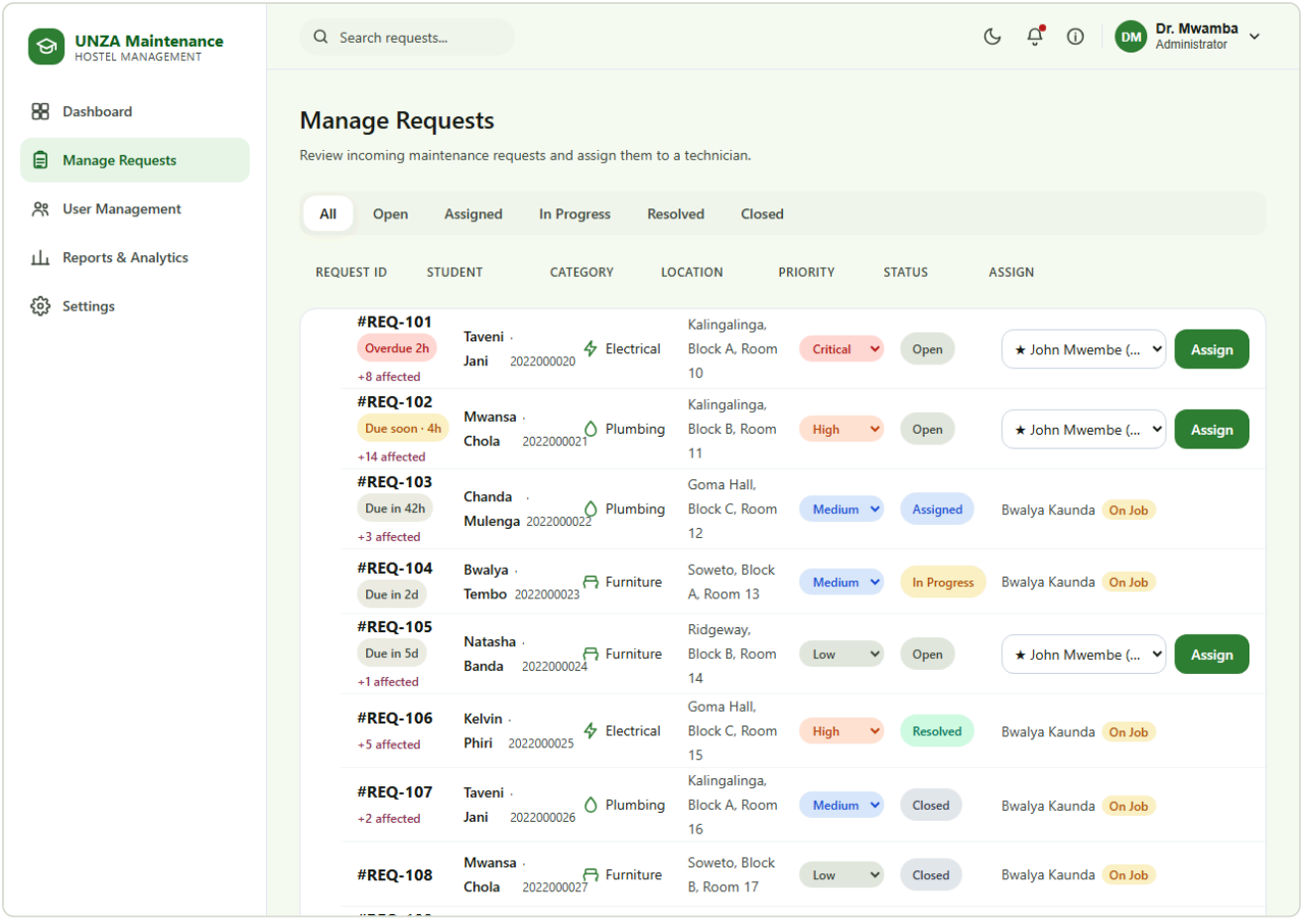


Dark mode



Animated category ring and resolution trend, plus an **Overdue Requests** tile.

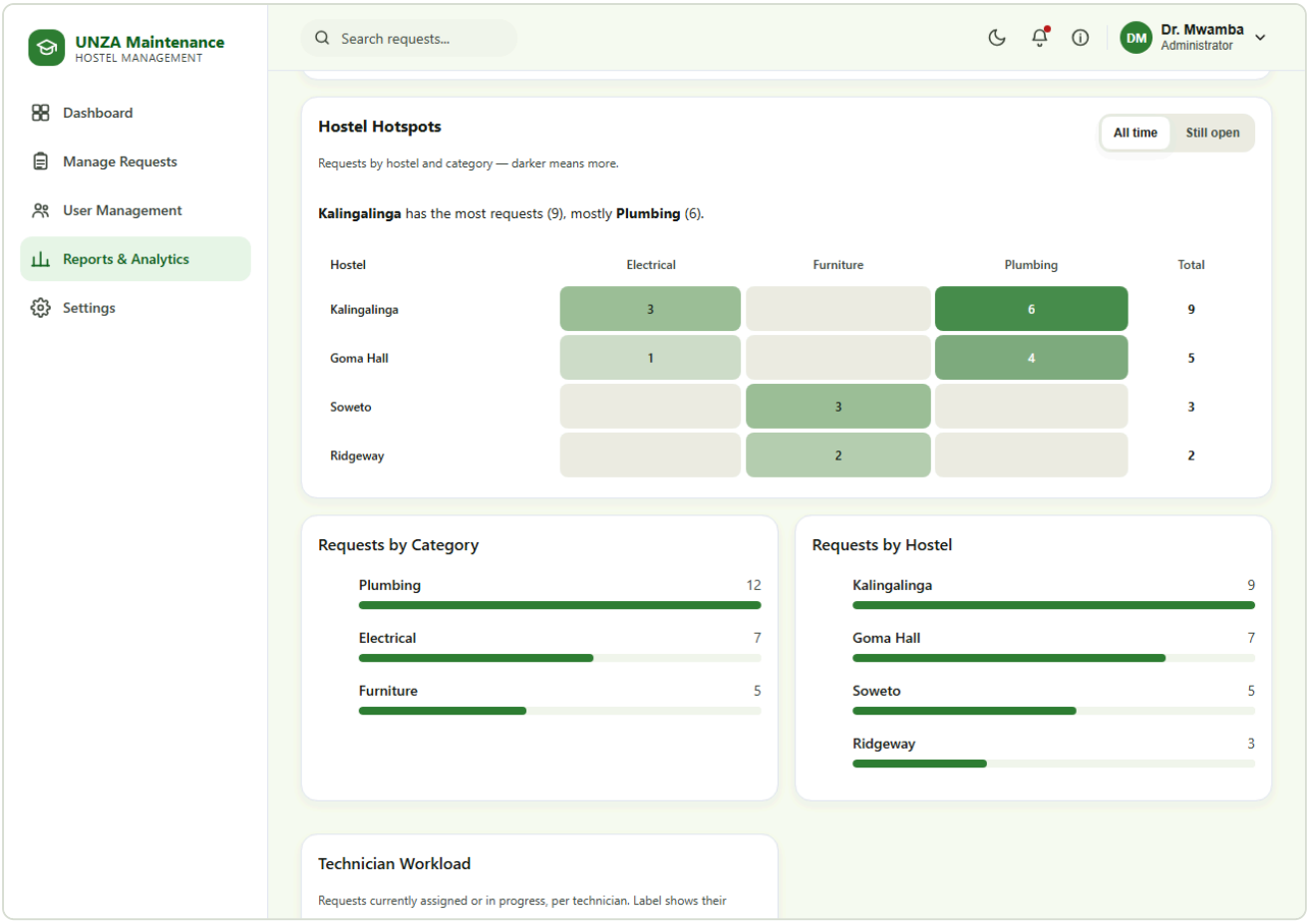
Manage requests



- **SLA badges**: *Overdue 2h*, *Due soon · 4h*, *Due in 2d*.

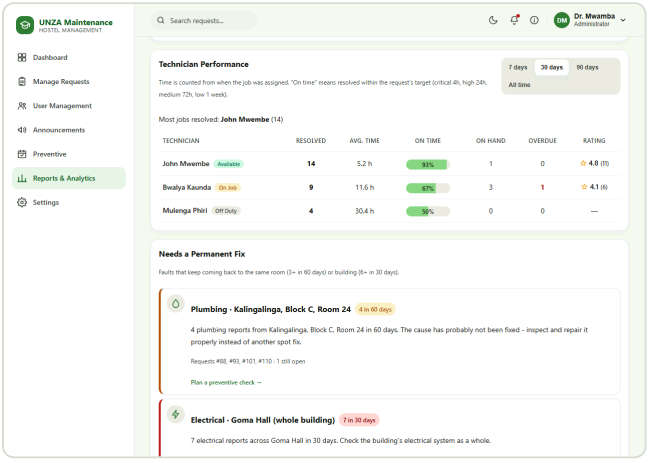
- **"+8 affected"** = students who upvoted the same issue.
- ★ **Recommended technician** (available, least busy) is preselected: one tap to assign.

Reports & Analytics — hostel hotspots

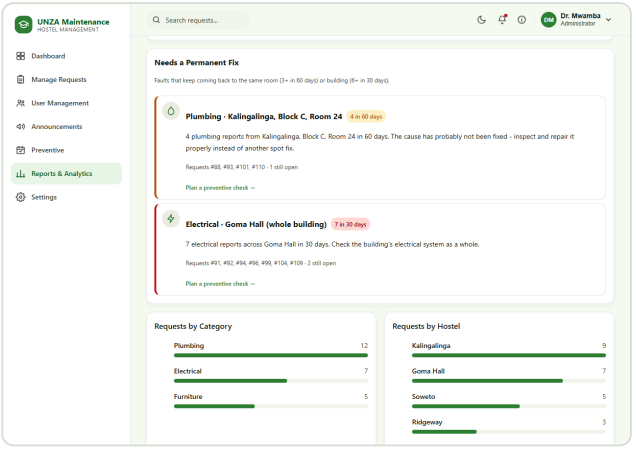


The heatmap answers *"where should maintenance go this week?"*. Reports also has a **Technician Performance** table and a **Needs a Permanent Fix** card (below), can be exported to CSV/PDF, and a weekly summary is emailed on Monday mornings.

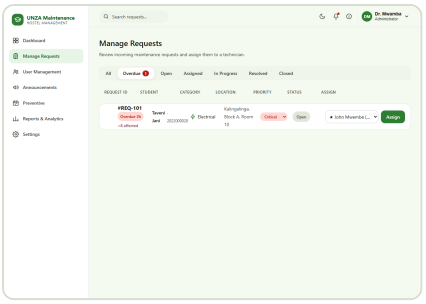
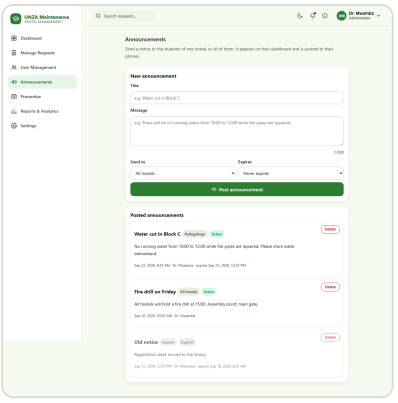
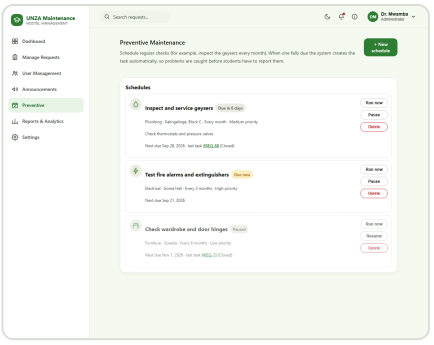
Technician Performance



Needs a Permanent Fix

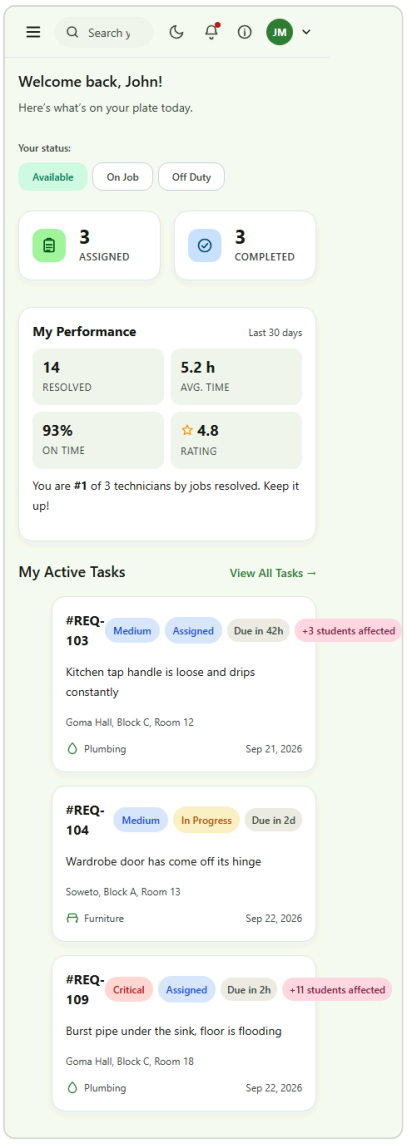
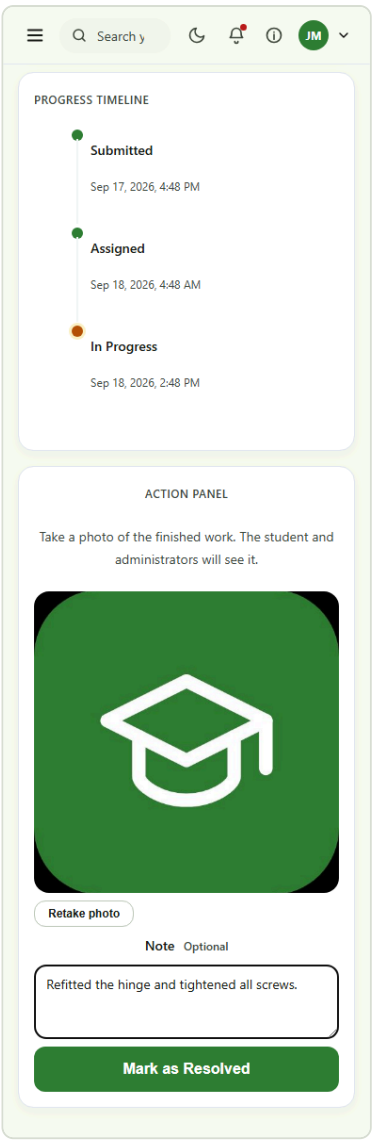


- **Technician Performance** ranks every technician (resolved, avg. time, on-time rate, jobs on hand, overdue, rating) over a chosen window.
- **Needs a Permanent Fix** flags faults repeating at the same room (3+/60 days) or hostel (6+/30 days), each with a **"Plan a preventive check"** link.

Overdue tab	Announcements	Preventive Maintenance
		

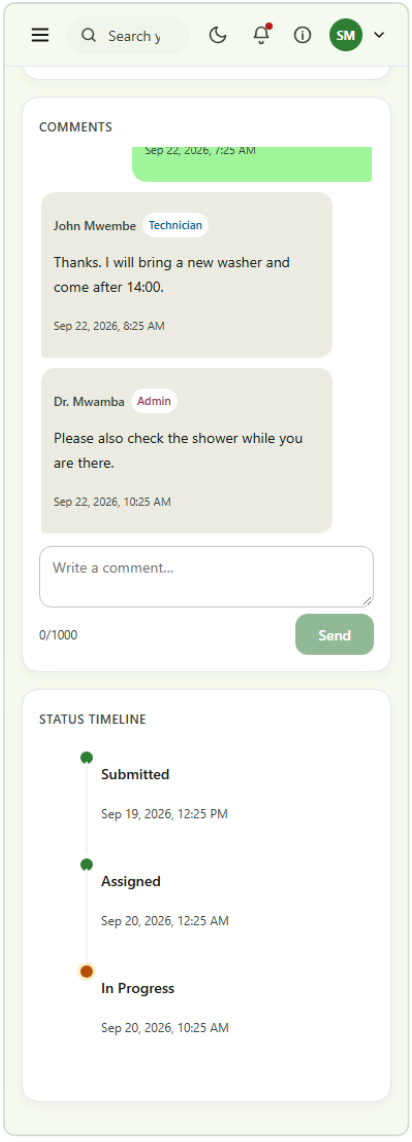
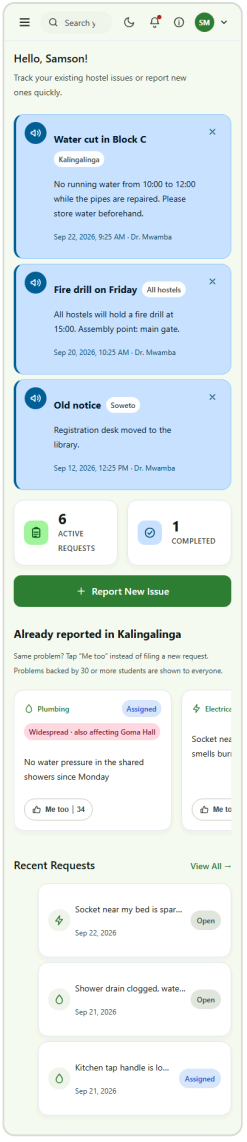
- **Manage Requests → Overdue tab** lists everything past its SLA deadline in one place; the dashboard's Overdue tile links straight into it.
- **Announcements** posts a notice to one hostel or everyone, with an optional expiry — shown as a banner on the student dashboard and pushed to their phones.
- **Preventive Maintenance** schedules recurring checks (e.g. "inspect the geysers monthly"); the system creates the maintenance request automatically when one falls due, so a schedule flows through assignment and proof of work exactly like a student report.

Technician

Task list, with My Performance	Proof of work
	

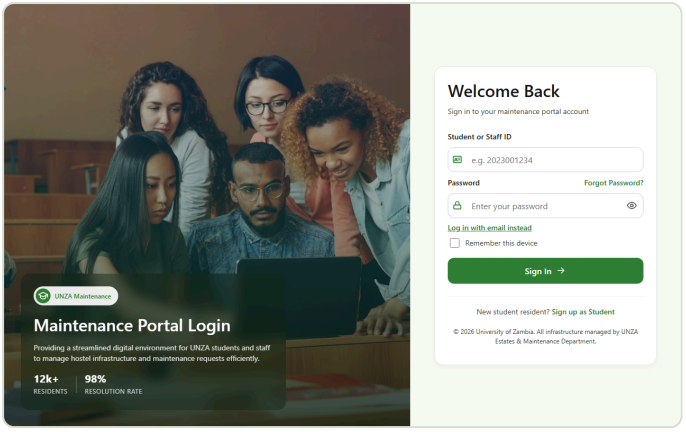
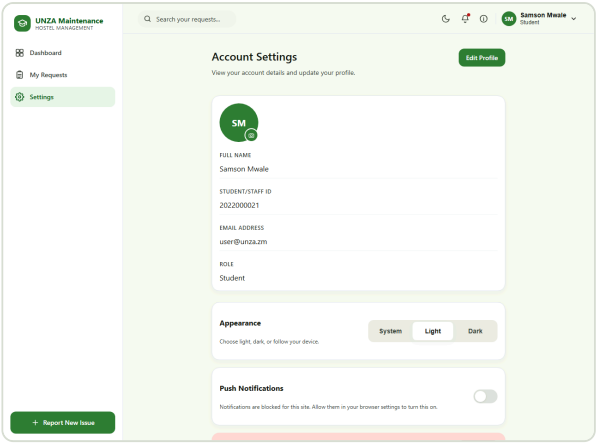
- **My Performance** — the technician's own 30-day scorecard: resolved, average time, on-time rate, rating, and rank among colleagues.
- Task cards show a **"+N students affected"** badge when several students backed the same issue with "Me too", so widely-felt problems get fixed first.
- When resolving, a technician can optionally **take a photo** (camera on a phone; a live preview on a computer) **or choose one from files**, plus a note. The image is compressed in the browser before upload. The photo can be made mandatory by setting `REQUIRE_RESOLUTION_PROOF=true`.

Communication and community

Comments on a request	Widespread issues (30+ upvotes)
	

- Any request has a **Comments** thread — only the reporting student, the assigned technician and administrators can see or post ("Which tap is it?").
- An issue backed by **30 or more "Me too" votes** is shown to students in every hostel, not just where it was reported, labelled "*Widespread*" — because a fault that size is unlikely to be confined to one building.

Everyone

Login (ID-based)	Settings
	

Login uses the **student/staff ID**; the server resolves it to the email that Supabase Auth needs. Settings hold **push notifications** and **light/dark/system** theme.

---

3. Suggested live-demo script (≈ 10 minutes)

1. **Student (phone-sized window).** Open the dashboard → point at the *Already reported* strip and an announcement banner → tap *Me too* on one issue.
2. **Report an issue.** Choose a hostel + category → the duplicate warning appears. Type "the socket near my bed is sparking" → the suggestion jumps to *Critical* → *Apply* → submit.
3. **Administrator (second window).** The new request appears **without a refresh** (live update) with a toast. Show the *Overdue* tab, the SLA badges and the *+N affected* badge. Assign the ★ recommended technician in one tap.
4. **Student window** updates live: status → *Assigned*. Post a **comment** asking which tap it is — it appears for the technician too.
5. **Technician.** Open the task → *Start Task* → add a photo (camera or file) and a short note → *Mark as Resolved*. Point at **My Performance** on the dashboard.
6. **Student** sees *Work Completed* with the photo and note; rate it.
7. **Administrator** → **Reports.** Show the heatmap, **Technician Performance**, **Needs a Permanent Fix** (→ *Plan a preventive check*), export CSV, and the *Email me the summary* button.
8. **Administrator** → **Announcements.** Post a notice to one hostel; it lands on the student's dashboard. → **Preventive.** Show a schedule and *Run now*.
9. **Show two "PWA" moments**: toggle dark mode; switch the browser to offline and draft a request (it queues), then go online and watch it sync. Mention install-to-home-screen.

Have these ready beforehand: a student, an administrator and a technician login, the SQL migrations applied, and at least a few requests in different states.

---

4. Requirements coverage

Requirement (from the project brief)	Delivered by
Auth: registration, login, roles	ID-based login resolved server-side; three roles enforced on server and UI
Student submits a request (category, description, optional photo)	Request form with photo upload, hostel/room, priority, AI suggestion
Lifecycle `Open → Assigned → In Progress → Resolved → Closed`	Enforced in `requests.service.js`; invalid transitions rejected
Administrator views and assigns requests	Manage Requests + request detail, smart technician recommendation
Technician views assigned work and updates status	My Tasks, task detail, proof of work
Student tracks status in real time	Server-Sent Events + status timeline
Status history on every transition	`status_history` table; drives the timeline and analytics
Analytics (average resolution time, workload)	Reports & Analytics, dashboard, heatmap, CSV/PDF export, weekly email
Role-based access control everywhere	`requireRole` + service ownership checks + RLS
PWA: manifest, service worker, offline drafting, sync on reconnect	Installable, cache strategies, IndexedDB draft queue
Non-functional: usability, performance, security, maintainability	See ARCHITECTURE.md §8, §11, §12

Beyond the brief: upvotes (with campus-wide visibility), SLA timers with overdue alerts and reminders, push notifications, live updates, AI triage, camera-capture proof of work, dark mode, hostel heatmap, weekly email, admin announcements, per-request comments, recurring-issue detection, technician performance, and preventive-maintenance scheduling.

---

5. Likely questions — and honest answers

Why an Express API instead of letting the React app call Supabase directly? Because access control and business rules must not depend on the browser. Centralising them in the server means one place to enforce and test them, and no database keys ship to the client. RLS then acts as a second line of defence.

Why three layers on the server? Each layer has one job and only calls downward. That is why the SLA rule, the lifecycle rules and the technician ranking are pure, unit-tested functions, and why adding a feature rarely touched more than one file per layer.

How do you stop a student reading someone else's request? `authenticate` verifies the JWT; the request service checks ownership (student must own it, technician must be assigned) and returns 403 otherwise; RLS backs it up at the database. The UI hiding a link is not what protects the data.

What happens if the AI is wrong — or unavailable? It is advisory only: the student must tap *Apply*, and nothing is saved by the suggestion. Without an API key, or on any error, a transparent keyword engine answers. A safety rule forces hazards (fire, sparking, flooding) to *critical* on both engines.

Why Server-Sent Events rather than WebSockets or Supabase Realtime? Updates only flow server → client; SSE is plain HTTP with automatic reconnect. Supabase Realtime would have required the client to talk to Supabase, which the architecture forbids. Events carry only an id and a message; the client refetches through the normal permission-checked API.

How does offline work, and what are its limits? The service worker caches the app shell; a new request written offline is stored in IndexedDB and submitted when the browser fires `online`. I implemented this in the page instead of Background Sync because Background Sync is not supported on iOS or Firefox. Limit: only *new requests* can be drafted offline; other actions need a connection.

Is it actually fast? Pages render instantly from a stale-while-revalidate cache and refresh in the background; the entry bundle is ≈ 79 kB gzipped with pages and the chart library split out; hashed assets are served from the service-worker cache. On the free host the first request after idle can be slow (cold start), which a scheduled health-check ping avoids.

What are the weaknesses? (1) The 70 integration tests fake the database, so they prove the routes → services → repositories wiring and access control, but not Postgres/RLS itself (that is checked in Supabase and by manual UAT); the front-end browser checks are not yet an automated suite. (2) Live updates, the token cache and the rate limiter are in memory, so they assume a single server instance. (3) Optional integrations (AI, email, push) need configuration. See ARCHITECTURE.md §13 — being upfront about these is deliberate.

What would you build next? Integration tests with a real test database, a shared store (Redis) for multi-instance scaling, letting a student edit or cancel their own request before it's assigned, and an SMS fallback for students without data.

How do you know the SLA/overdue logic is right? It is a pure function with unit tests for every state and boundary (on track, at risk, overdue, done), used identically by the API, the overdue monitor and the weekly email. Any change is made once and tested once.

Is anything AI-generated in the code? Be ready to answer this honestly for your institution's policy: this project was built with an AI coding assistant. What matters in a viva is that you understand every decision in this document and can explain the rules, the layering and the trade-offs — so read ARCHITECTURE.md until you can explain each section without notes.